

DNN Partitioning and Assignment for Distributed Inference in SGX Empowered Edge Cloud

Yuepeng Li*, Deze Zeng^{✉*}, Lin Gu[†], Song Guo[‡], Albert Y. Zomaya[§]

*School of Computer Science, China University of Geosciences, Wuhan, China

[†]School of Computer Science and Technology, Huazhong University of Science and Technology, China

[‡]Department of Computing, The Hong Kong Polytechnic University, Hong Kong

[§]School of Computer Science, The University of Sydney, Australia

[✉]Engineering Research Center of Natural Resource Information Management and Digital Twin Engineering Software, Ministry of Education, Wuhan 430074, China

Abstract—Distributed Deep Neural Network (DNN) inference is a promising technology to explore the distributed resources in edge cloud to realize edge intelligence. Meanwhile the inherent resource sharing nature of edge cloud infrastructure also raises serious concerns on security and privacy. Software Guard Extensions (SGX) emerges as a potential hardware-level solution but its limited secure memory (i.e., enclave page cache) imposes new challenges, especially in contrast to memory-hungry DNN models. A task's performance will be severely affected when its memory footprint is beyond the enclave page cache size, due to expensive secure page swapping. In this case, how to appropriately partition a DNN model and assign the partitions to distributed edge servers to efficiently utilize edge resources for fast secure inference becomes a challenging problem. In this paper, we first show that this problem is NP-hard. We further propose a Memory-aware Distributed Inference Acceleration (MEDIA) algorithm, whose guaranteed approximation ratio is also formally analyzed. We have implemented a prototype system and applied some well-known representative DNN models to evaluate MEDIA's performance. Through extensive experiments, we verify the efficiency of MEDIA by the fact that it reduces the inference time by 19.5%-38.1% in comparison with state-of-the-art approaches.

Index Terms—DNN, SGX, Distributed inference

I. INTRODUCTION

Edge intelligence has experienced revolutionary development during the past years. It is anticipated that edge intelligence will facilitate almost every aspect of our daily lives by provisioning various applications such as smart healthcare [1], autonomous driving [2], and advanced video analytics [3]. Deep learning, empowered by deep neural network (DNN), is one of the key technologies in edge intelligence. DNN model, albeit highly potential, is notorious for its extremely high resource consumption as it is both memory-intensive and computation-intensive. For example, the widely adopted DNN model GoogLeNet requires about 155.9MB memory and 1.5 billion multiply-add operations, and VGG16 even needs 1.16GB memory and 19.6 billion multiply-add operations [4]. It is widely recognized that edge servers are relatively resource-constrained. This makes the situation even

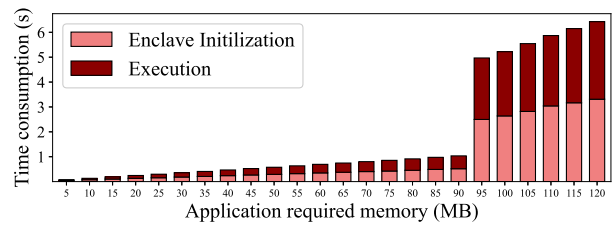


Fig. 1. The enclave program overhead under different memory requirements

worse as the DNN resource requirements are usually beyond the capacity of edge servers [5]. Fortunately, distributed DNN recently has emerged as a compelling solution to tackle the contradiction between resource supply and demand in both DNN training and inference [6].

Edge cloud based distributed DNN inference, by making each edge server handle a part of inference task within its ability, therefore is of great significance to edge intelligence [5], [7], [8]. Much recent effort has been devoted to optimizing the distributed DNN inference performance in edge cloud, objective at inference acceleration [7], request throughput promotion [8] and cost minimization [5]. Besides, the security and privacy of DNN inference is another important issue drawing growing attention, due to the resource sharing of edge cloud. For example, a malicious tenant co-locating on the same server can acquire sensitive data via using side-channel attacks [9] or injection attacks [10]. Even the untrustworthy infrastructure providers may also intentionally thwart the DNN model inference process by injecting fake data [11]. In this regard, many solutions, such as homomorphic encryption [12], garbled circuits [13], have been proposed. Unfortunately, these cryptology based solutions are not only performance inefficient but also with limited application scope [14].

Hence, Software Guard Extensions (SGX) [15], as a hardware-level protection technology, has been advocated to many security-critical applications [16]–[19], including intelligent applications with DNN inference [14], [20]. SGX builds a special secure memory region called *Enclave Page Cache (EPC)*, in which the loaded code and data are protected from other untrusted programs (including the operation system) during execution. However, the secure memory capacity, i.e.,

This research was supported by the NSF of China (No. 62172375), Provincial Key Research and Development Program of Hubei (No.2023BAB065). Deze Zeng (deze@cug.edu.cn) is the corresponding author.

EPC size, of SGX is limited to 128MB in the latest products [15]. Once the memory usage of a task exceeds the EPC size, it will suffer from critical performance degradation. We conducted a preliminary experiment to assess such degradation by gradually increasing the memory requirement of a character counter application. We sent different sizes of character strings into EPC, and loop over the characters to find the occurrence of specific characters. The task execution time on different memory requirements is reported in Fig. 1. We can see sudden performance degradation when the memory requirement exceeds 93MB. Before and after that point, the task execution time almost linearly increases with the memory requirement as normal. This is because, when an application's memory footprint saturates the EPC, excessive secure *EPC page swapping* between the EPC and unprotected memory will be incurred [21]. Meanwhile, although EPC is 128MB, around 35MB must be allocated to metadata [20]. Note that EPC page swapping is expensive, costing up to hundreds of thousands of cycles for each swapping, as it does not only entail data transfer to and from EPC, but also involves page encryption and integrity check for security guaranteeing [15].

Considering the large memory footprint of DNN models, the capacitated EPC becomes a catastrophe to deep learning based applications requiring DNN inference in SGX empowered edge cloud. Hence, how to partition a DNN model and assign the partitions to edge servers to efficiently utilize the edge resources for fast and secure inference emerges as a new challenge. Too fine-grained partitioning can avoid EPC page swapping, but may lead to massive inter-partition communication traffic. On the contrary, coarse-grained partitioning reduces the inter-partition communication overhead, yet results in high EPC page swapping overhead and performance degradation. Therefore, instead of simply treating EPC size as a hard limit, appropriate partitioning and assignment decisions should be made to balance the partition execution time and inter-partition communication time for overall inference acceleration. In this paper, we are motivated to study how to partition and assign a DNN model to an SGX empowered edge cloud to accelerate distributed DNN inference. The main contributions of this paper are:

- We study how to partition a DNN model and appropriately assign the partitions to distributed edge servers with SGX, aiming to minimize the inference time. The problem is formally stated and proved to be NP-hard. To our best knowledge, we are the first to investigate distributed inference with the consideration of SGX characteristics.
- We propose a Memory-aware Distributed Inference Acceleration (MEDIA) algorithm that well balances the partition execution time in SGX and inter-partition communication time. The achievable guaranteed approximation ratio of MEDIA algorithm is also formally analyzed.
- We implement a prototype system based on DarkNet framework to evaluate the proposed MEDIA algorithm, and conduct extensive experiments on representative linear and nonlinear DNN models. The results show that our

algorithm can reduce the inference time by 19.5%-38.1% in comparison with state-of-the-art approaches.

The rest of this paper is organized as follows. Section II introduces some related work. In Section III, we introduce the system model and problem statement. Section IV presents MEDIA algorithm and corresponding theoretical analysis. Section V presents our prototype based evaluation results. Finally, we conclude this work in Section VI.

II. RELATED WORKS

Distributed DNN inference is a potential way to take full advantage of edge cloud for intelligent applications. Many existing distributed DNN inference solutions in the literature have targeted inference acceleration. For example, DNN surgery [8] and CRA [22] split a DNN model into two partitions, one processed at the edge and another to the cloud. The former takes a graph-theoretic approach to find the minimum cut, and the latter designs a Markov based algorithm to obtain a near-optimal solution. Mohammed et al. [23] propose DINA, which includes a fine-grained DNN partition policy and matching game based computation offloading method allowing multiple DNN partitions to be processed locally by end devices or offloaded to multiple powerful nodes for inference acceleration. These solutions mainly target at general processors without awareness of SGX characteristics, especially the EPC size limitation. Some also consider memory capacity and constrain a partition within it. Note that, although EPC size is limited, the memory footprint could still exceed it, at the cost of certain performance degradation.

Meanwhile, distributed DNN inference offers the opportunity to utilize distributed resources, but it also raises the security and privacy problem, especially in resource sharing infrastructure like edge cloud. Pioneering researchers have applied various means to guarantee DNN inference security, such as fully or semi-homomorphic encryption [12], [24], [25] and multi-party computation [13], [26], [27]. However, these cryptography based methods suffer from severe performance degradation and limited application scope because of the complex cryptographic operations involved. To facilitate secure DNN inference with low overhead, SGX is widely advocated and many recent studies have tried to address the EPC size limitation problem. SECURETF [28] uses model reduction technique to reduce the size of DNN models and improves inference performance in SGX. Vessels [20] overcomes the EPC limitation through optimizing memory reclaiming during the DNN inference process. Occlumency [14] divides a DNN model into several partitions and executes them sequentially on a server to cope with the limitation. However, existing studies usually treat EPC size as a hard limit and mainly concentrate on a single server to avoid excessive page swapping. In distributed DNN inference, we argue that certain page swapping is allowable provided that the performance degradation could be compensated by the reduction of inter-partition communication overhead. Unfortunately, SGX aware distributed DNN inference optimization is still under-investigated, and this motivates our work.

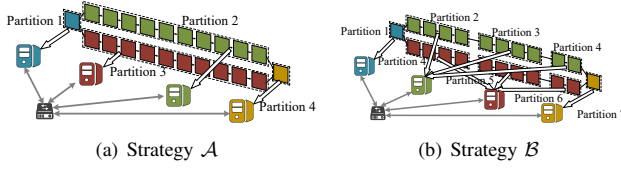


Fig. 2. AlexNet model partitioning and assignment strategies

TABLE I
INFERENCE TIME BY DIFFERENT STRATEGIES

Strategy	Inference time	
	With SGX	Without SGX
Strategy A	16.1s	5.2s
Strategy B	9.1s	6.4s
Single server	20.4s	11.1s

III. SYSTEM MODEL AND PROBLEM STATEMENT

A. Motivations

The limited EPC size deeply affects the performance of DNN inference on SGX empowered edge servers. To take a close look into such impact, we use widely used AlexNet with 22 layers as an example and evaluate the inference time in an edge cloud consisting of 4 servers equipped with Intel Celeron G4930. Two cases, i.e., with and without SGX, are considered. In either case, two different DNN partitioning and assignment strategies, as shown in Fig. 2, are applied and the results on inference time are reported in Table I.

First of all, we can see that applying SGX to guarantee security indeed is at the cost of long inference time. In any case, the one with SGX always takes more inference time than the one without SGX. Fortunately, we can see that the performance loss could be effectively compensated by exploiting more servers. As shown in Table I, even when SGX is enforced for security guaranteeing, Strategy B still achieves better performance than the single server case without SGX.

We can also clearly see that different partitioning and assignment strategies lead to different inference time. What's especially interesting is that Strategy B speeds up the inference by $1.77\times$ over Strategy A when SGX is enforced, but instead, it even performs worse than Strategy A in the case without SGX. Obviously, the main difference between the two strategies is on the partition. In the case with SGX, Strategy A has relatively large partitions (e.g., Partition 2 and Partition 3 both with 10 layers), whose memory footprint exceeds the EPC size, resulting in excessive EPC page swapping and long inference time. On the contrary, in the case without SGX, too fine-grained partitioning may incur high inter-partition communication overhead, and hence long inference time.

This example indicates that, to pursue fast secure inference in SGX empowered edge cloud, we should well balance the page swapping overhead and communication overhead by taking both EPC size and DNN model structure into consideration.

B. Problem Statement and Analysis

Next, we formally state the partitioning and assignment problem toward minimal inference time.

1) *SGX Empowered Edge Cloud*: In this paper, we consider an edge cloud comprising a set $\mathbb{N} = \{1, 2, 3, \dots, N\}$ of SGX empowered servers, whose EPC is in size of E . Any two servers can communicate with each other, either directly or indirectly. Without loss of generality, $B_{m,n}$ denotes the reciprocal communication bandwidth between server m and n .

Recall Fig. 1, a sudden performance degradation happens when the memory requirement exceeds EPC size. Before and after the point, the task execution time almost linearly increases with the memory usage. Hence, the computing power of server n can be expressed as a function of memory usage as

$$F_n(\alpha_n) = \begin{cases} f_n, & \alpha_n \leq E, \\ f'_n, & \alpha_n > E, \end{cases} \quad (1)$$

where α_n denotes the memory usage on server n .

2) *DNN Model Partitioning*: As we have known, a DNN model can be represented as a DAG $\mathbb{G} = (\mathbb{V}, \mathbb{E})$, where $\mathbb{V}$ is the set of vertexes representing the DNN layers and $\mathbb{E}$ is the set of edges about the dependency between layers. A layer $v \in \mathbb{V}$ is indivisible and must be processed on one server. An edge $(u, v) \in \mathbb{E}$ indicates that u should be processed before v , and u feeds its output to v . The data size transferred from layer u to v is indicated as $d(u, v)$, which is determined by the DNN model structure. Given the structure of a DNN model, the workload and memory requirement of a layer $v \in \mathbb{V}$ can be estimated as $w(v)$ and $m(v)$, respectively.

In distributed DNN inference, we partition a large model into a set $\mathbb{P}$ of partitions, each of which consists of a subset of layers (i.e., $P \subseteq \mathbb{V}, \forall P \in \mathbb{P}$) to be executed on one server. The partitions must cover all the layers in the model. That is,

$$\cup_{P \in \mathbb{P}} P = \mathbb{V}. \quad (2)$$

Besides, a layer $l \in \mathbb{V}$ must be included in one and only one partition. Hence, we have

$$P \cap P' = \emptyset, \forall P, P' \in \mathbb{P}, P \neq P'. \quad (3)$$

The workload of a partition P can be calculated as $w(P) = \sum_{p \in P} w(p)$. Similarly, the memory footprint of partition P can be calculated as $m(P) = \sum_{p \in P} m(p), \forall P \in \mathbb{P}$. The new edge set $\mathbb{L}$ represents the dependency between different partitions. We use $d(P', P)$ to denote the data size transferred from partitions P' to P . $\mathbb{L}$ and $d(P', P)$ can inherit from $\mathbb{E}$ and $d(u, v)$, respectively.

3) *Partition Assignment*: The partitions $\mathbb{P}$ should be assigned to edge servers, which collaboratively process the assigned partitions to complete the whole inference task. We define binary variables $x_{P,n} \in \{0, 1\}$ to indicate whether partition P is assigned to server n (i.e. $x_{P,n} = 1$) or not (i.e. $x_{P,n} = 0$). Each partition $P \in \mathbb{P}$ must be executed on one server. That is,

$$\sum_{n \in \mathbb{N}} x_{P,n} = 1, \forall P \in \mathbb{P}. \quad (4)$$

With respect to the dependency relationship, a partition is ready to start only when all its predecessors have finished and all the required input data have been received. Let $RT(P)$ and $FT(Q)$ be the ready time and finish time of partitions $P \in \mathbb{P}$ and $Q \in \mathbb{P}$, respectively. Accordingly, we have

$$RT(P) \geq FT(Q) + \frac{x_{Q,m}x_{P,n}d(Q,P)}{B_{m,n}}, \quad (5)$$

$$\forall P \in \mathbb{P}, Q \in Pred(P), m \in \mathbb{N}, n \in \mathbb{N},$$

where $Pred(P)$ is the predecessor set of partition P .

At most one partition can be executed on an SGX empowered server at a time. If there are multiple partitions assigned to one server, they will be executed sequentially, i.e.,

$$FT(P) \leq ST(Q) \parallel FT(Q) \leq ST(P), \forall Q, P \in \mathbb{P}, \quad (6)$$

where $ST(\cdot)$ represents the start time of a partition. This explains why we distinguish start time from ready time, which actually should satisfy the following relationship

$$ST(P) \geq RT(P), \forall P \in \mathbb{P}. \quad (7)$$

Once a partition P is started, its finish time can be estimated by its workload and the assigned server's computing power as

$$FT(P) = ST(P) + \sum_{n \in \mathbb{N}} \frac{x_{P,n}w(P)}{F(m(P))}, \forall P \in \mathbb{P}. \quad (8)$$

By summing up the above, the distributed DNN inference problem to minimize inference time in SGX empowered edge cloud can be formulated as

$$\min \{ \max_{P \in \mathbb{P}} \{ FT(P) \} \}$$

s.t. : (2) – (7).

Obviously, this is a nonlinear integer programming problem, which is non-trivial to solve. Let us consider a special case when EPC is sufficient and all partitions have been determined. The thing we need to do is to assign the partitions to appropriate edge servers to minimize inference time. Essentially, it is equivalent to assigning tasks of a DAG to heterogeneous processors to minimize the makespan. This problem has been formulated into a flow-shop scheduling problem and proved to be NP-hard [29]. Therefore, the DNN model partitioning and assignment problem, as a general case, is NP-hard.

IV. MEMORY-AWARE DISTRIBUTED DNN INFERENCE ACCELERATION ALGORITHMS

To tackle the computation complexity, we design **ME**Memory-aware **D**istributed **I**nference **A**cceleration (**MEDIA**) algorithm in this section, which mainly consists of partitioning phase and assignment phase.

A. MEDIA Partitioning

In the partitioning phase, we divide a DNN model into a group of partitions and restructure the partitions into a coarse DAG, with the consideration of EPC characteristics (i.e., EPC capacity and performance degradation after exceeding the capacity). Based on multilevel graph partitioning methods [30],

Algorithm 1: Edge Selection for MEDIA Partitioning

Input: Model graph $\mathbb{G} = (\mathbb{V}, \mathbb{E})$; The level of vertexes in $\mathbb{V}$; The weight of edges in $\mathbb{E}$.
Output: The subset of edges $\mathbb{M}$.

```

1 Set  $\mathbb{M} \leftarrow \emptyset, \mathbb{P} \leftarrow \emptyset$ 
2 for  $u \in \mathbb{V}$  following the increasing order of level do
3   for  $v \in Succ(u)$  following the priority on edges do
4     if  $(|Pred(v)| \neq 1)$  and  $(|Succ(u)| \neq 1)$  then
5       continue
6      $\mathbb{M} \leftarrow \mathbb{M} \cup \{(u, v)\}$ 
7     for  $w \in succ(u)$  do
8       if  $L(u) = L(w) - 1$  and there is an
9          $(w', w) \in \mathbb{M}$  then
10         $\mathbb{M} \leftarrow \mathbb{M} \setminus \{(u, v)\}$ 
11      end
12    end
13  end
14 end
```

we design our MEDIA partitioning strategy, which consists of the following two stages.

- First, we find a set of edges where the vertexes along each edge can be collapsed.
- Next, we generate partitions via collapsing the vertexes along these edges with the consideration of EPC characteristics.

The details of these two stages are presented in Algorithm 1 and Algorithm 2, respectively.

1) *Edge Selection:* As we assign the coarse graph to edge servers in the unit of partition, the partition assignment decisions are related to the estimated ready time and finish time of partitions. According to (5), it is impossible to estimate the time if the graph is cyclic. Therefore, we should first find a subset of edges in $\mathbb{G}$, i.e., $\mathbb{M} = \{(u_1, v_1), (u_2, v_2), \dots, (u_k, v_k)\} \subseteq \mathbb{E}$ to ensure that the coarse graph $\mathbb{G}_{\mathbb{M}}$ to be created is acyclic.

Lemma 1 *The coarse graph $\mathbb{G}_{\mathbb{M}}$ is acyclic, if $\mathbb{M} = \{(u_1, v_1), (u_2, v_2), \dots, (u_k, v_k)\}$ satisfies the following constraints [31].*

- *Constraint 1:* $\forall i \in \{1, 2, \dots, k\}, Succ(u_i) = \{v_i\}$, or $Pred(v_i) = \{u_i\}$,
- *Constraint 2:* $\forall i \neq j \in \{1, 2, \dots, k\}, (u_i, v_j) \notin \mathbb{E}$, or $L(u_i) + 1 \neq L(v_j)$,

where $Succ(u_i)$ represents the immediate successors of u_i and $L(v)$ is the topological order of vertex v in $\mathbb{G}$.

The topological order could be obtained via any graph traverse algorithm such as depth-first search (DFS).

Following Lemma 1, we propose our edge selection algorithm in Algorithm 1 to produce edge set $\mathbb{M}$, based on which an acyclic graph $\mathbb{G}_{\mathbb{M}}$ can be constructed. We traverse the vertexes of graph $\mathbb{G}$ via DFS as shown in line 2. For each vertex u , if its successor v does not satisfy *Constraint 1* in Lemma 1, vertexes u and v cannot be collapsed. Then, we

Algorithm 2: Memory-aware MEDIA Partitioning**Input:** The subset of edges $\mathbb{M}$ **Output:** DNN partitioning results $\mathbb{P}$.

```

1 for  $(u, v) \in \mathbb{M}$  do
2   if Both vertexes in  $(u, v)$  are not collapsed then
3     if Check  $(\{u\}, \{v\})$  then
4       Generate a new partition  $P = \{u, v\}$ 
5        $\mathbb{P} \leftarrow \mathbb{P} \cup P$ 
6     end
7   else if Both vertexes in  $(u, v)$  are collapsed then
8     Find the partition  $P \in \mathbb{P}$  includes  $u$ 
9     Find the partition  $P' \in \mathbb{P}$  includes  $v$ 
10    if Check  $(P, P')$  then
11       $P \leftarrow P \cup P'$ 
12    end
13  else if Only one vertex in  $(u, v)$  is not collapsed
    then
14    Find partition  $P \in \mathbb{P}$  with the collapsed vertex
15    Let  $w$  denote the uncollapsed vertex in  $(u, v)$ 
16    if Check  $(P, \{w\})$  then
17       $P \leftarrow P \cup \{w\}$ 
18    end
19  end
20 end
21 Function Check  $(P, P')$  :
22   if  $\bar{T}(P \cup P') \leq \bar{T}(P) + \bar{T}(P') + \bar{T}(P')$  or
      $m(P \cup P') \leq E$  then return true
23   return false
24 end

```

move to its successor v , as shown in line 4. If v satisfies *Constraint 1*, edge (u, v) will be added into subset $\mathbb{M}$ in line 5. We further check if vertex u satisfies *Constraint 2* or not. For any other successor of u , say w , if there is an edge $(w', w) \in \mathbb{M}$ and $L(u) = L(w) - 1$, adding edge (u, v) into $\mathbb{M}$ will contradict *Constraint 2*. In this case, edge (u, v) should be removed from $\mathbb{M}$, as shown in line 6 to line 10. We repeat this process until all edges (u, v) satisfying both *Constraint 1* and *Constraint 2* are added into $\mathbb{M}$.

2) *Graph Partition*: Next, we will construct an acyclic coarse graph $\mathbb{G}_{\mathbb{M}}$ based on the selected edge set $\mathbb{M}$ via collapsing the vertexes into partitions. The major concern here is on the EPC. Basically, we greedily collapse dependent vertexes into the same partition, subject to the EPC capacity, to avoid inter-partition communication. However, the newly formed partition's memory requirement may be beyond the EPC size. Note that MEDIA does not treat the EPC size as a hard limit and allows overranging, provided that the incurred performance degradation is tolerable. That is, MEDIA collapses two partitions into one if the achieved inference time is less than executing them separately, despite the overranging.

We estimate the execution time of a partition as

$$\bar{T}(P) = \frac{w(P)}{\sum_{n \in \mathbb{N}} F_n(m(P))/|\mathbb{N}|}. \quad (9)$$

During the inference process, certain data must be transferred between dependent partitions. The inter-partition communication time between two dependent partitions, say P and P' , can be estimated as

$$\bar{T}(P, P') = \frac{d(P, P')}{\sum_{m \in \mathbb{N}} \sum_{n \in \mathbb{N}} B_{m,n}/|\mathbb{N}|}. \quad (10)$$

Now, we can define function Check (P, P') to assess whether two partitions P and P' should be collapsed into one or not, as shown in lines 21-24 in Algorithm 2. If the total memory requirement is within the EPC size, i.e., $m(P \cup P') \leq E$, without doubt that we should construct a large partition by collapsing them. Otherwise, if $\bar{T}(P \cup P') \leq \bar{T}(P) + \bar{T}(P, P') + \bar{T}(P')$, we still put them into one.

Then, we can apply the function Check $()$ and traverse the edges in $\mathbb{M}$ to construct partitions. We will meet the following three situations.

- If both vertexes of edge (u, v) are not collapsed, we apply Check $()$ to decide whether they can be collapsed into one partition or not. If true, we will generate a new partition containing u and v (lines 2-6).
- If both vertexes of edge (u, v) have been collapsed, we find the partition P with u and the partition P' with v . We will collapse P and P' into one large partition if function Check $()$ tells us it is beneficial to do so. (lines 7-12).
- If only one vertex of edge (u, v) has been collapsed, we first find the partition P including the collapsed vertex, and apply function Check $()$ to check whether another vertex can be collapsed into P or not (lines 13-19).

After traversing all the edges in $\mathbb{M}$, we can get the partitions of a DNN model and obtain the corresponding coarse DAG $\mathbb{G}_{\mathbb{M}} = \{\mathbb{P}, \mathbb{L}\}$.

B. MEDIA Assignment

After obtaining $\mathbb{G}_{\mathbb{M}}$, we next should assign the partitions $\mathbb{P}$ into appropriate edge servers toward the minimum inference time. During partition assignment, first of all, the topological order of $\mathbb{G}_{\mathbb{M}}$ must be guaranteed. That is, the predecessors of a partition (say P) should be assigned before P . For example, the partition with the input layer must be assigned at the very beginning. Secondly, the partition with high resource requirements should be assigned as early as possible so as to find a server with sufficient resources. With respect to the above two issues, we define the assignment priority of a partition P as

$$Priority(P) = \max_{P' \in succ(P)} \{\bar{T}(P) + \bar{T}(P, P') + Priority(P')\}. \quad (11)$$

Lemma 2 *The decreasing order of priority provides a topological ordering of partitions in the graph.*

Proof: From the definition in (11), for any edge $(P, P') \in \mathbb{L}$ in the graph $\mathbb{G}_{\mathbb{M}}$, we always have $Priority(P) \geq Priority(P')$. The topological order can be guaranteed by the decreasing order of priority. ■

Algorithm 3: MEDIA Assignment Algorithm

```

1 for  $P \in \mathbb{P}$  do
2   Calculate the priority of partition  $P$  following (11)
3 end
4 for  $P \in \mathbb{P}$  following decreasing order of priorities do
5    $\mathbb{T} \leftarrow \emptyset$ 
6   for  $n \in \mathbb{N}$  do
7     Calculate  $FT(P)$  on server  $n$  via (5)-(7)
8      $\mathbb{T} \leftarrow \mathbb{T} \cup \{FT(P)\}$ 
9   end
10  Find the minimum  $FT(P)$  from set  $\mathbb{T}$  and assign
    the partition  $P$  to the corresponding server  $s$ 
11 end

```

By Lemma 2, we can conclude that the partitions with higher priority should be assigned earlier to ensure their topological dependency. Based on such principle, we propose a prioritized MEDIA Assignment algorithm, as shown in Algorithm 3. We first calculate the priority of each partition in line 2. Then, we traverse all the partitions following the decreasing order of partition priority. We choose the unassigned partition P with the highest priority and calculate its expected finish time on each server following (5)-(7), in lines 6 to 9. After that, partition P will be assigned to the server that achieves the lowest finish time $FT(P)$ in line 10. The above procedure repeats until all partitions are assigned.

C. Theoretical Analysis of MEDIA

Next, we analyze the gap between the inference time obtained by MEDIA and the optimal solution, which are denoted as O_{media} and O_{opt} , respectively. For tractability, we discrete the inference time O_{media} into a number of time slots. These time slots can be categorized into two subsets $\mathbb{A}$ and $\mathbb{B}$, where busy time slots $\mathbb{A}$ is defined as the set of all time slots when all edge servers are busy, and idle time slots $\mathbb{B}$ includes all time slots when at least one edge server is idle.

We first consider the idle time slots $\mathbb{B}$ represented by a disjoint union of q open time intervals (b_{l_t}, b_{r_t}) , i.e., $\mathbb{B} = (b_{l_1}, b_{r_1}) \cup (b_{l_2}, b_{r_2}) \cup \dots \cup (b_{l_q}, b_{r_q})$, where $b_{l_1} < b_{r_1} < b_{l_2} < b_{r_2} < \dots < b_{l_q} < b_{r_q}$.

Lemma 3 We can always extract a chain of partitions as $\mathbb{C} : P_l \rightarrow P_{l-1} \rightarrow \dots \rightarrow P_1$ from the coarse graph that satisfies

$$\sum_{j=1}^q (b_{r_j} - b_{l_j}) \leq \sum_{k=1}^l \left(\frac{F_{\max}}{F_{\min}} T_{\min}(P_k) \right) + \sum_{k=1}^{l-1} T_{\max}^{trans}, \quad (12)$$

where $T_{\max}^{trans}$ is the maximum data transmission time in chain $\mathbb{C}$, $T_{\min}(P_k)$ is the minimum execution time of partition P_k in the edge cloud, and $F_{\max}$ and $F_{\min}$ denote the maximum and minimum computing power of edge servers.

Proof: Let P_1 denote the last partition assigned by MEDIA algorithm. There are three cases regarding the start time of P_1 , i.e.,

- $ST(P_1) \leq b_{l_1}$.

- $ST(P_1) \in \mathbb{B}$. That is, there exists an integer h , $h \leq q$, such that $b_{l_h} \leq ST(P_1) \leq b_{r_h}$.
- $ST(P_1) \in \mathbb{A}$ but $ST(P_1) > b_{l_1}$. That is, there exists an integer h , $h \leq q-1$, such that $b_{r_h} \leq ST(P_1) \leq b_{l_{h+1}}$ or $b_{r_q} \leq ST(P_1)$.

For the above three cases, we can always add one or more partitions to the left of the chain to satisfy (12). In detail, we construct $\mathbb{C}$ for each case as follows.

Case 1: In this case, the partition P_1 by itself constitutes a chain that satisfies (12).

Case 2: If $ST(P_1)$ falls into $\mathbb{B}$, we can find another partition P_2 to construct chain $\mathbb{C}$ following two sub-cases independently as below.

SUB-CASE 1: If P_1 does not become ready until $ST(P_1)$, indicating that $ST(P_1) = RT(P_1)$, we simply add the last finished predecessor of P_1 as P_2 into the chain.

SUB-CASE 2: If P_1 's ready time is earlier than $ST(P_1)$, i.e., $RT(P_1) < ST(P_1)$, by the definition of $\mathbb{B}$, there must be an idle server $n \in \mathbb{N}$ during (b_{l_h}, b_{r_h}) where $RT(P_1) < b_{r_h}$, indicating that P_1 is ready when n is idle. Intuitively, P_1 can start earlier on n but actually MEDIA does not assign P_1 to server n to make it start earlier. According to MEDIA, there must exist another assignment satisfying $FT(P_1) \leq FT^n(P_1)$ where $FT^n(P_1)$ be the finish time of P_1 assigned to server n . Let P_2 denote the last finished predecessor of P_1 . It is obvious that

$$FT^n(P_1) \leq FT(P_2) + T_{\max}^{trans} + T_{\max}(P_1), \quad (13)$$

where $T_{\max}(P_1) = \frac{F_{\max}}{F_{\min}} T_{\min}(P_1)$ denotes the maximum execution time of partition P_1 in the edge cloud. Then, we can construct the chain via inserting partition P_2 into the left of P_1 and derive that

$$T_{\max}^{trans} + \frac{F_{\max}}{F_{\min}} T_{\min}(P_1) \geq FT(P_1) - FT(P_2). \quad (14)$$

Case 3: Same as *Case 2*, we insert the last finished immediate predecessor of P_1 into the chain, and we can still obtain (14).

When a new partition is inserted into the chain, it will meet one of the three cases mentioned above. If *Case 1* happens, we are done. Otherwise, we need to add more partitions in the way mentioned in *Case 2* or *Case 3*. Such process repeats until *Case 1* happens. Finally, chain $\mathbb{C}$ satisfying (12) is successfully constructed. ■

Hence, we can derive the idle time of MEDIA as follows.

Lemma 4 In MEDIA algorithm, the total idle time of all the servers satisfies

$$\sum_{n \in \mathbb{N}} T_{idle}(n) \leq |\mathbb{N}| \left(\frac{F_{\max}}{F_{\min}} O_{opt} + \sum_{k=1}^{l-1} T_{\max}^{trans} \right), \quad (15)$$

where $T_{idle}(n)$ denotes the idle time of server n .

Proof: It is obvious that the total idle time on one server is smaller than the total time in set $\mathbb{B}$. Therefore, we have

$$\sum_{n \in \mathbb{N}} T_{idle}(n) \leq |\mathbb{N}| \sum_{j=1}^q (b_{r_j} - b_{l_j}). \quad (16)$$

With Lemma 3 and (16), we can derive

$$\sum_{n \in \mathbb{N}} T_{idle}(n) \leq |\mathbb{N}| \left(\sum_{k=1}^l \left(\frac{F_{\max}}{F_{\min}} T_{\min}(P_k) \right) + \sum_{k=1}^{l-1} T_{\max}^{trans} \right). \quad (17)$$

In addition, many existing studies [32], [33] have already proved that, for any DAG, its optimal makespan is longer than the makespan of any chain extracted from it. Hence, we can further conclude that

$$\sum_{k=1}^l T_{\min}(P_k) \leq O_{opt}. \quad (18)$$

Now, we can derive (15) with (17) and (18). ■

Then, we consider the busy time of MEDIA.

Lemma 5 *In MEDIA algorithm, the total busy time satisfies*

$$\sum_{n \in \mathbb{N}} T_{busy}(n) \leq |\mathbb{N}| \frac{F_{\max}}{F_{\min}} O_{opt}, \quad (19)$$

where $T_{busy}(n)$ denotes the busy time of server n .

Proof: A server $n \in \mathbb{N}$ is considered as busy whenever there is a partition being executed. Hence, the left-hand of (19) is equal to the total time of executing all assigned partitions. That is,

$$\sum_{n \in \mathbb{N}} T_{busy}(n) = \sum_{n \in \mathbb{N}} T_{exe}(n), \quad (20)$$

where $T_{exe}(n)$ denotes the time of executing partitions on server n . Obviously, $\sum_{n \in \mathbb{N}} T_{exe}(n)$ cannot exceed $\frac{F_{\max}}{F_{\min}} \sum_{P \in \mathbb{P}} T_{\min}(P)$. Hence, we have

$$\sum_{n \in \mathbb{N}} T_{busy}(n) \leq \frac{F_{\max}}{F_{\min}} \sum_{P \in \mathbb{P}} T_{\min}(P). \quad (21)$$

Existing work [34] has proved that $\sum_{P \in \mathbb{P}} T_{\min}(P) \leq |\mathbb{D}| O_{opt}$. Thus, (19) can be derived from (21). ■

With Lemma 4 and Lemma 5, we can obtain the upper bound of O_{media} as follows.

Theorem 1 *The inference time of MEDIA always satisfies*

$$O_{media} \leq 2 \frac{F_{\max}}{F_{\min}} O_{opt} + \sum_{k=1}^{l-1} T_{\max}^{trans}.$$

Proof: As O_{media} consists of the busy and idle time, it can be expressed as

$$O_{media} = \sum_{n \in \mathbb{N}} (T_{idle}(n) + T_{busy}(n)) / |\mathbb{N}|. \quad (22)$$

According to Lemma 4 and Lemma 5, (22) can be rewritten as

$$\begin{aligned} O_{media} &= \sum_{n \in \mathbb{N}} (T_{idle}(n) + T_{busy}(n)) / |\mathbb{N}| \\ &\leq 2 \frac{F_{\max}}{F_{\min}} O_{opt} + \sum_{k=1}^{l-1} T_{\max}^{trans}. \end{aligned} \quad \blacksquare$$

Therefore, our MEDIA algorithm can achieve inference time no worse than $2 \frac{F_{\max}}{F_{\min}} O_{opt} + \sum_{k=1}^{l-1} T_{\max}^{trans}$.

TABLE II
CPU MODELS OF SGX-ENABLED SERVERS FOR EXPERIMENTS

Server ID	CPU Type	Frequency
Server 1	Intel Core i5-11600	4.60GHz
Server 2	Intel Core i3-10100	3.60GHz
Server 3	Intel Core i5-6500	3.20GHz
Server 4	Intel Celeron G4930	3.20GHz

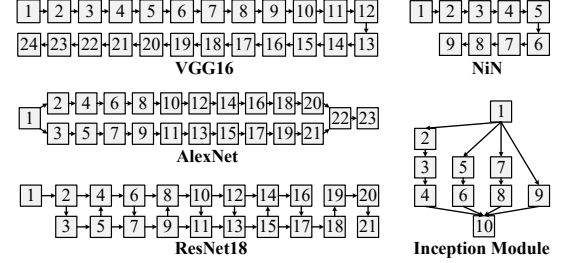


Fig. 3. The structure of linear and nonlinear DNN models

V. PERFORMANCE EVALUATION

To evaluate the performance of MEDIA, we have implemented a prototype system based on DarkNet. In this section, we report our prototype implementation, experiment settings, and performance evaluation results.

A. Implementation and Experimental Setup

1) *Prototype system implementation:* The DarkNet cannot be ported in SGX directly because system calls and CUDA are forbidden in SGX. To build the prototype system, we re-implement all system calls and the modules relying on CUDA, such as information logging, fully-connected layer module and convolution layer module. By such means, we successfully port the DarkNet framework into SGX and realize the prototype system.

2) *Prototype system setup:* As shown in Table II, the experiment environment consists of 4 servers equipped with SGX 2.7 environment and Ubuntu 18.04 operating system to emulate an SGX empowered edge cloud. These servers are connected to each other and the average bandwidth between any two servers is set as 10Mbps via wonder shaper in default.

3) *DNN models:* To evaluate the adaptability of MEDIA, we adopt 6 DNN models in different structures, including 2 linear models as NiN and VGG16, and 4 nonlinear models as ResNet18, AlexNet, InceptionV3 and InceptionV4. Their structures are shown in Fig. 3.

4) *Comparison algorithms:* We compare MEDIA against state-of-the-art schemes, including Occlumency (OCC) [14], Dynamic Adaptive DNN surgery (DADS) [8] and Distributed INference Acceleration (DINA) [23]. We run each experiment 10 times and report the average inference time with an error bar representing standard deviation.

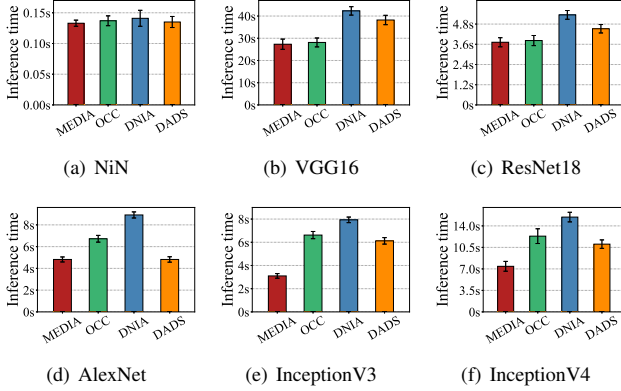


Fig. 4. The inference time of different DNN models by different algorithms

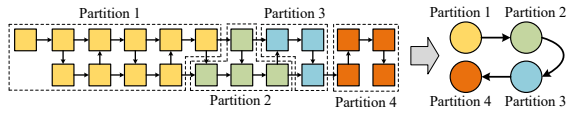


Fig. 5. The partition result of ResNet18 obtained by MEDIA

B. The Inference Time of Different DNN Models

We first evaluate the inference time and memory usage of MEDIA on all 6 DNN benchmarks mentioned above. The results are shown in Fig. 4 and Fig. 6, respectively.

It can be observed that MEDIA achieves the lowest inference time among all schemes, as shown in Fig. 4. The high efficiency of our MEDIA is validated by the fact that it significantly reduces the inference time by 19.5%, 38.1% and 29.4% in comparison with OCC, DNIA and DADS, respectively. We attribute this inference time speedup to better partitioning and assignment decisions made by MEDIA. Instead of simply treating EPC size as a hard limit, MEDIA allows a partition's memory footprint to exceed EPC size, and balances the EPC page swapping overhead and the inter-partition communication overhead for overall inference acceleration. It is noticeable that the inference time of NiN by all four algorithms is close. This is mainly because the total memory requirement of NiN model is about 78.4MB, smaller than EPC size. As a result, *EPC page swapping* will not happen during the execution of NiN and there is not much optimization space for partitioning and assignment.

Another interesting finding is that OCC algorithm sometimes achieves comparable inference time to MEDIA in Figs. 4(a)~4(c). Note that the advantage of MEDIA over OCC is that MEDIA can well leverage distributed edge servers and enable parallel DNN partition execution. However, when the structure of DNN model is linear, all partitions must be executed one by one sequentially. In our experiments, the coarse DAGs of linear DNN models NiN and VGG16 are still linear. Although ResNet18 itself is nonlinear, it is partitioned and restructured into a coarse DAG in linear structure, as shown in Fig. 5. By OCC or MEDIA, all partitions of

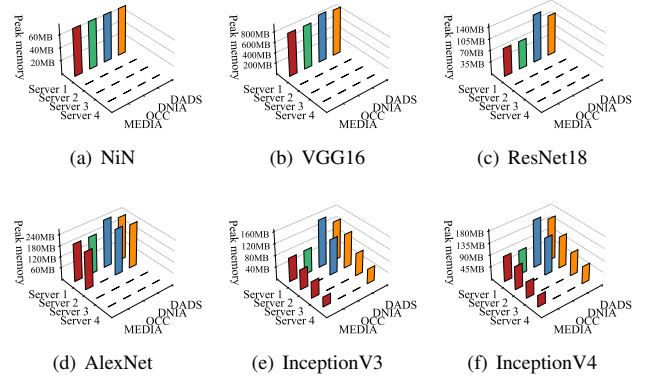


Fig. 6. The peak memory usage of different algorithms on different servers

these three models will be assigned to the server with the highest computing power and executed sequentially. Hence, the inference time of OCC and MEDIA is comparable. While, for most non-linear models (e.g., AlexNet, InceptionV3 and InceptionV4 in Figs. 4(d)-4(f)), their restructured coarse DAGs are still non-linear. The partitions in non-linear coarse DAG can be assigned to different servers and executed in parallel. In this case, compared with OCC, MEDIA significantly reduces the inference time by 28.3%, 47.6% and 39.8% for AlexNet, InceptionV3 and InceptionV4, respectively.

To further validate the fact that our MEDIA and OCC effectively take EPC size into consideration, we present the peak memory usages of all four algorithms in Fig. 6. It can be seen that both MEDIA and OCC ensure the peak memory usage of each partition around 93MB for most DNN models. However, in InceptionV3 and InceptionV4 cases, the peak memory usage of MEDIA, as almost up to 120MB, exceeds the EPC size. Remember that MEDIA does not treat EPC size as a hard limit, instead it allows EPC overranging to trade execution performance degradation with inter-partition communication minimization for overall performance optimization. One may also notice that the peak memory usage of VGG16 is as large as 836.4MB. This is because that some indivisible layers of VGG16 are large. For example, layer 20 in VGG16 (see Fig. 3) itself requires more than 800MB memory. In such circumstances, the secure EPC page swapping is inevitable. The optimization space left for MEDIA becomes limited. Nevertheless, MEDIA still shows more advantages than other algorithms in most DNN models.

C. The Impact of Server Number

Next, we present the DNN inference time with different number of servers. In this experiment, we add SGX-enabled servers with different computing power into the edge cloud by the following order: 2 servers with Intel Celeron G4930, 4 servers with Intel Core i5-5600, 1 server with Intel Core i3-10100 and 1 server with Intel Core i5-11600 to test the algorithms' sensitivity on heterogeneous computing power.

As shown in Fig. 7, the inference time shows as a non-increasing trend with the increase of server number in all cases,

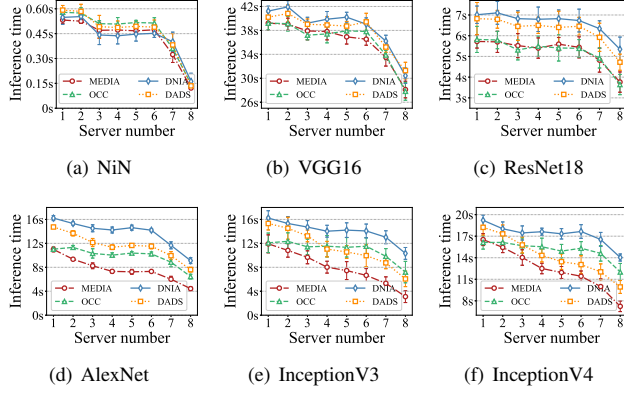


Fig. 7. The inference time on different server numbers

and our MEDIA always achieves the best performance. On average, the inference time of MEDIA is reduced by 11.6%, 27.8% and 17.3% in comparison with OCC, DNIA and DADS, respectively. This is because MEDIA always tries to fully utilize distributed computing resources in edge cloud and the possible parallelism of DNN models to efficiently speed up the DNN inference. We also observe that the inference time of all algorithms in Figs. 7(a)~7(c) does not change too much when the server number grows from 3 to 6. As we have known, the NiN, VGG16 and ResNet18 models are all linear. Their partitions are executed on the most powerful server. While, the 4 newly added servers are in the same type, and hence the inference time remains almost the same from 3 servers to 6 servers. However, when more powerful servers 7 and 8 are added, they are selected for execution and the inference time decreases, as shown in Figs. 7(a)~7(c).

In addition, from Figs. 7(d)~7(f), we further observe that InceptionV3 and InceptionV4 are more sensitive to the server number and computing power than AlexNet. For example, the inference time of InceptionV3 and InceptionV4 by MEDIA is reduced by 68.3% and 58.2%, respectively, when the server number is increased from 1 to 8 in Fig. 7(e) and Fig. 7(f). But when it comes to AlexNet, the inference time of MEDIA is only reduced by 49.3%, as shown in Fig. 7(d). It is because that the InceptionV3 and InceptionV4 have more independent layers than AlexNet, as shown in Fig. 3. Hence, the partitions of AlexNet are executed parallelly on up to 2 servers, while InceptionV3 and InceptionV4 can well leverage all servers.

D. The Impact of Network Bandwidth

Finally, we investigate the effect of network bandwidth via increasing the bandwidth from 1Mbps to 10Mbps, and the experiment results are shown in Fig. 8. Once again, MEDIA still always achieves the best performance, speeding up the inference by $1.25\times$, $1.85\times$ and $1.52\times$ over OCC, DNIA and DADS, respectively. Besides, for NiN, VGG16 and ResNet18, the inference time does not change too much in Figs. 8(a)~8(c). As we have known, all the partitions of these models are assigned to the most powerful server, without data

transferring. Hence, the inference time will not be affected by network bandwidth.

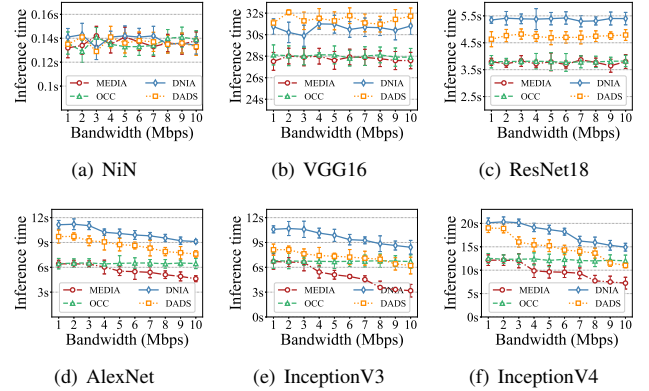


Fig. 8. The inference time on different average network bandwidths

For AlexNet, InceptionV3 and InceptionV4, we can see that the inference time of MEDIA, DNIA and DADS decreases with the increase of network bandwidth, as shown in Figs. 8(d)~8(f). This is because higher bandwidth implies lower inter-partition communication time, resulting in faster inference. Furthermore, we discover another phenomenon that, for InceptionV3 and InceptionV4, the inference time by MEDIA is reduced by almost 50% when the average network bandwidth is increased from 1Mbps to 10Mbps in Fig. 8(d) and Fig. 8(e), but only 27% for AlexNet in Fig. 8(d). As we have known, compared with AlexNet, MEDIA produces more partitions in the cases of InceptionV3 and InceptionV4 to exploit the distributed computing resources. This implies higher inter-partition data transferring needs and therefore more sensitive to network bandwidth. Consequently, InceptionV3 and InceptionV4 benefit more than AlexNet from the increase of network bandwidth.

What's more, from Figs. 8(d)~8(f), we find that the inference time of MEDIA is not affected too much by network bandwidth, and even is close to the inference time of OCC, when the bandwidth is below 3Mbps. When the bandwidth is too small, MEDIA will assign all partitions to one powerful server to avoid long inter-partition communication time. In this case, MEDIA achieves similar performance with OCC. However, as the network bandwidth increases, MEDIA is able to well balance execution time and communication time for overall inference acceleration, and therefore becomes advantageous over OCC.

VI. CONCLUSION

In this paper, we investigate how to accelerate distributed DNN inference in SGX empowered edge cloud. Being aware of the capacitated EPC, we formulate the DNN model partitioning and partition assignment problem for minimal inference time into a nonlinear integer programming form, and prove it as NP-hard. We further propose MEDIA algorithm that first reconstructs the DNN model into an acyclic coarse graph to well balance EPC page swapping overhead

(i.e., computation performance degradation) and inter-partition communication overhead, and then assigns the partitions to edge servers to efficiently utilize distributed resources. The guaranteed approximation ratio of MEDIA is theoretically analyzed. We practically implement MEDIA and build a prototype system. Via extensive comparison with state-of-the-art schemes, the results on commonly-used DNN models show that MEDIA achieves equivalent performance to OCC [14] on linear models and greatly outperforms all existing schemes on non-linear models. Averagely, it can significantly reduce the inference time by 19.5%-38.1%.

REFERENCES

- [1] X. Zhang, Y. Wang, S. Lu, L. Liu, L. Xu, and W. Shi, "OpenEI: An Open Framework for Edge Intelligence," in *Proceedings of IEEE International Conference on Distributed Computing Systems (ICDCS)*, pp. 1840–1851, 2019.
- [2] H. Zhou, W. Li, Z. Kong, J. Guo, Y. Zhang, B. Yu, L. Zhang, and C. Liu, "Deepbillboard: Systematic Physical-World Testing of Autonomous Driving Systems," in *Proceedings of IEEE/ACM International Conference on Software Engineering (ICSE)*, pp. 347–358, 2020.
- [3] L. Zeng, X. Chen, Z. Zhou, L. Yang, and J. Zhang, "CoEdge: Cooperative DNN Inference with Adaptive Workload Partitioning Over Heterogeneous Edge Devices," *IEEE/ACM Transactions on Networking (TON)*, vol. 29, no. 2, pp. 595–608, 2020.
- [4] A. Mahmood, M. Bennamoun, S. An, and F. Sohel, "Resfeats: Residual Network based Features for Image Classification," in *Proceedings of IEEE International Conference on Image Processing (ICIP)*, pp. 1597–1601, 2017.
- [5] Z. Xu, L. Zhao, W. Liang, O. F. Rana, P. Zhou, Q. Xia, W. Xu, and G. Wu, "Energy-Aware Inference Offloading for DNN-Driven Applications in Mobile Edge Clouds," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 32, no. 4, pp. 799–814, 2020.
- [6] J. Du, X. Zhu, M. Shen, Y. Du, Y. Lu, N. Xiao, and X. Liao, "Model Parallelism Optimization for Distributed Inference Via Decoupled CNN Structure," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 32, no. 7, pp. 1665–1676, 2020.
- [7] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Distributed Deep Neural Networks over the Cloud, the Edge and End Devices," in *Proceedings of IEEE International Conference on Distributed Computing Systems (ICDCS)*, pp. 328–339, 2017.
- [8] C. Hu, W. Bao, D. Wang, and F. Liu, "Dynamic Adaptive DNN Surgery for Inference Acceleration on the Edge," in *Proceedings of IEEE International Conference on Computer Communications (INFOCOM)*, pp. 1423–1431, 2019.
- [9] J. Wei, Y. Zhang, Z. Zhou, Z. Li, and M. A. Al Faruque, "Leaky DNN: Stealing Deep-learning Model Secret with GPU Context-switching Side-channel," in *Proceedings of IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 125–137, 2020.
- [10] Y. Liu, L. Wei, B. Luo, and Q. Xu, "Fault Injection Attack on Deep Neural Network," in *Proceedings of IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp. 131–138, 2017.
- [11] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li, "Manipulating Machine Learning: Poisoning Attacks and Countermeasures for Regression Learning," in *Proceedings of IEEE Symposium on Security and Privacy (S&P)*, pp. 19–35, 2018.
- [12] B. Reagen, W.-S. Choi, Y. Ko, V. T. Lee, H.-H. S. Lee, G.-Y. Wei, and D. Brooks, "Cheetah: Optimizing and Accelerating Homomorphic Encryption for Private Inference," in *Proceedings of IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 26–39, 2021.
- [13] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A Cryptographic Inference Service for Neural Networks," in *Proceedings of USENIX Security Symposium (USENIX Security)*, pp. 2505–2522, 2020.
- [14] T. Lee, Z. Lin, S. Pushp, C. Li, Y. Liu, Y. Lee, F. Xu, C. Xu, L. Zhang, and J. Song, "Oclumency: Privacy-preserving Remote Deep-learning Inference Using SGX," in *Proceedings of ACM International Conference on Mobile Computing and Networking (MobiCom)*, pp. 1–17, 2019.
- [15] V. Costan and S. Devadas, "Intel SGX Explained," *IACR Cryptol. ePrint Arch.*, vol. 2016, no. 86, pp. 1–118, 2016.
- [16] S. Arnaudov, B. Trach, F. Gregor, T. Knauth, A. Martin, C. Priebe, J. Lind, D. Muthukumar, D. O'keefe, M. L. Stillwell, D. Goltzsche, D. Eysers, R. Kapitza, P. Pietzuch, and C. Fetzer, "SCONE: Secure Linux Containers with Intel SGX," in *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 689–703, 2016.
- [17] C.-C. Tsai, D. E. Porter, and M. Vij, "Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX," in *Proceedings of USENIX Annual Technical Conference (ATC)*, pp. 645–658, 2017.
- [18] M. Orenbach, P. Lifshits, M. Minkin, and M. Silberstein, "Eleos: ExitLess OS Services for SGX Enclaves," in *Proceedings of ACM European Conference on Computer Systems (EuroSys)*, pp. 238–253, 2017.
- [19] C. Priebe, K. Vaswani, and M. Costa, "Enclavedb: A secure database using sgx," in *Proceedings of IEEE Symposium on Security and Privacy (S&P)*, pp. 264–278, 2018.
- [20] K. Kim, C. H. Kim, J. J. Rhee, X. Yu, H. Chen, D. Tian, and B. Lee, "Vessels: Efficient and Scalable Deep Learning Prediction on Trusted Processors," in *Proceedings of ACM Symposium on Cloud Computing (SoCC)*, pp. 462–476, 2020.
- [21] A. Biondo, M. Conti, L. Davi, T. Frassetto, and A.-R. Sadeghi, "The Guard's Dilemma: Efficient Code-Reuse Attacks Against Intel SGX," in *Proceedings of USENIX Security Symposium (USENIX Security)*, pp. 1213–1227, 2018.
- [22] W. He, S. Guo, S. Guo, X. Qiu, and F. Qi, "Joint DNN Partition Deployment and Resource Allocation for Delay-Sensitive Deep Learning Inference in IoT," *IEEE Internet of Things Journal*, vol. 7, no. 10, pp. 9241–9254, 2020.
- [23] T. Mohammed, C. Joe-Wong, R. Babbar, and M. Di Francesco, "Distributed Inference Acceleration with Adaptive DNN Partitioning and Offloading," in *Proceedings of IEEE International Conference on Computer Communications (INFOCOM)*, pp. 854–863, 2020.
- [24] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy," in *Proceedings of International Conference on Machine Learning (ICML)*, pp. 201–210, 2016.
- [25] G. Lloret-Talavera, M. Jorda, H. Servat, F. Boemer, C. Chauhan, S. Tomishima, N. N. Shah, and A. J. Pena, "Enabling homomorphically encrypted inference for large dnn models," *IEEE Transactions on Computers (TC)*, vol. 71, pp. 1145–1155, 2021.
- [26] N. Kumar, M. Rathee, N. Chandran, D. Gupta, A. Rastogi, and R. Sharma, "CrypTFlow: Secure TensorFlow Inference," in *Proceedings of IEEE Symposium on Security and Privacy (S&P)*, pp. 336–353, 2020.
- [27] A. S. Shamsabadi, A. Gascón, H. Haddadi, and A. Cavallaro, "Privedge: From Local to Distributed Private Training and Prediction," *IEEE Transactions on Information Forensics and Security (TIFS)*, vol. 15, pp. 3819–3831, 2020.
- [28] D. L. Quoc, F. Gregor, S. Arnaudov, R. Kunkel, P. Bhatotia, and C. Fetzer, "SECURETF: A Secure TensorFlow Framework," in *Proceedings of International Middleware Conference (Middleware)*, pp. 44–59, 2020.
- [29] J.-J. Hwang, Y.-C. Chow, F. D. Anger, and C.-Y. Lee, "Scheduling Precedence Graphs in Systems with Interprocessor Communication Times," *SIAM Journal on Computing*, vol. 18, no. 2, pp. 244–257, 1989.
- [30] N. Jafari, O. Selvitopi, and C. Aykanat, "Fast Shared-memory Streaming Multilevel Graph Partitioning," *Journal of Parallel and Distributed Computing (JPDC)*, vol. 147, pp. 140–151, 2021.
- [31] J. Herrmann, J. Kho, B. Uçar, K. Kaya, and Ü. V. Çatalyürek, "Acyclic partitioning of large directed acyclic graphs," pp. 371–380, 2017.
- [32] E. Bampis and A. V. Kononov, "Bicriteria Approximation Algorithms for Scheduling Problems with Communications Delays," *Journal of Scheduling*, vol. 8, no. 4, pp. 281–294, 2005.
- [33] D. R. Karger, C. Stein, and J. Wein, "Scheduling Algorithms," in *Algorithms and Theory of Computation Handbook*, ser. Chapman & Hall/CRC Applied Algorithms and Data Structures series. CRC Press, 1999.
- [34] J. Hwang, Y. Chow, F. D. Anger, and C. Lee, "Scheduling Precedence Graphs in Systems with Interprocessor Communication Times," *SIAM Journal on Computing*, vol. 18, no. 2, pp. 244–257, 1989.